Bölüm 13: NumPy Bölüm 1

SE 422 · Introduction to Data Science · Konu özeti

Kaynak slayt destesi: NumPy_part1
7 Kasım 2026

İçindekiler

Bölüm 13 · NumPy Bölüm 1 3

Bölümün amacı 3

Konu anlatımı 3

Python listesinin sorunu (Slayt 1–3) 3

ndarray'in farkı (Slayt 4) 3

Vektörleştirme ve SIMD (Slayt 5–6) 4

Ne kadar hızlı? Ne zaman değil? (Slayt 7–8) 4

N boyutlu dizi, eksenler ve kısıtlamalar (Slayt 9–10) 5

ndim, shape, size, dtype, itemsize (Slayt 11–13) 5

Dizi oluşturmak (Slayt 14–18) 6

1 boyutlu indeksleme ve dilimleme (Slayt 19–20) 7

Çok boyutlu indeksleme (Slayt 21–22) 7

reshape ve 3 boyut (Slayt 23–24) 7

View: listede kopya, NumPy'da pencere (Slayt 25–27) 8

Deep copy: .copy() (Slayt 28–29) 8

Sık yapılan hatalar 9

Kısa tekrar soruları 9

Cevaplar 10

Bölüm 13 · NumPy Bölüm 1

Bölümün amacı

Bu bölüm veri biliminin temel paketi **NumPy**'ı ve onun çekirdek veri yapısı **ndarray**'i öğretir. Sonunda şunları yapabilmelisin:

- Python listelerinin sayısal hesaplamadaki sorununu ve NumPy'ın neden hızlı olduğunu açıklamak
- Vektörleştirmeyi kullanmak; NumPy'ın ne zaman avantajlı olmadığını bilmek
- `ndim`, `shape`, `size`, `dtype`, `itemsize` özniteliklerini okumak
- `np.array`, `zeros`, `ones`, `empty`, `arange`, `linspace`, `eye` ile dizi oluşturmak
- 1, 2 ve 3 boyutlu dizileri indekslemek, dilimlemek ve `reshape` etmek
- **view** ile **copy** arasındaki farkı ve `.copy()`'nin ne zaman gerektiğini açıklamak

Konu anlatımı

Python listesinin sorunu (Slayt 1–3)

Python listesi esnek bir kaptır ve farklı tipte (heterojen) değerler tutabilir. Ama verinin kendisini değil, bellekte **dağınık duran nesnelere işaret eden pointer'ları** saklar. Liste üzerinde döngü kurulunca **her eleman için** pointer takip edilir, nesnenin tipi belirlenir ve ancak sonra işlem yapılır. Bu **ek yük** (overhead) büyük ve tekrarlı sayısal işlemlerde pahalıdır. NumPy bu boşluğu doldurmak için yaratıldı.

ndarray'in farkı (Slayt 4)

NumPy dizisi (**ndarray**) iki özellik taşır:

- **Homojendir:** tüm elemanlar aynı tiptedir, yani bellekte aynı miktarda yer kaplar.
- **Tek, bitişik bir bellek bloğunda** saklanır.

Bu düzen pointer takibi ve tip kontrolü yükünü kaldırır; daha önemlisi hesaplamaları **C veya Fortran** ile yazılmış, önceden derlenmiş koda devretmeyi sağlar.

```python
import numpy as np
a = np.array([1, 2, 3])
print(a, type(a).__name__, a.dtype)
```

ÇIKTI

```
[1 2 3] ndarray int64
```

Vektörleştirme ve SIMD (Slayt 5–6)

Vektörleştirme, elemanları döngüyle tek tek dolaşmak yerine işlemi **dizinin tamamı** üzerine yazmaktır. Vektörleştirilmiş kod derlenmiş altyapıyı tetikler; bu altyapı **SIMD** (Single Instruction, Multiple Data) ile tek bir komutu birden çok veriye aynı anda uygulayabilir. Ayrıca kod matematik gösterimine benzer, kısa ve okunaklıdır, daha az hata barındırır.

PYTHON

```python
import numpy as np
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])
print(a + b, a * 2, a ** 2)
liste = [1, 2, 3]
print([x * 2 for x in liste], liste * 2)   # listede * çoğaltma yapar, çarpma
↪ değil
```

ÇIKTI

```
[11 22 33] [2 4 6] [1 4 9]
[2, 4, 6] [1, 2, 3, 1, 2, 3]
```

Ne kadar hızlı? Ne zaman değil? (Slayt 7–8)

NumPy büyük veride listelerden **kat kat** (orders of magnitude) hızlıdır; bu yüzden Pandas, Scikit-learn ve tüm bilimsel Python ekosistemi NumPy'ı temel alır. **En iyi uygulama:** büyük veri üzerindeki açık döngüleri NumPy ile değiştir. Ancak **tek bir skaler** ya da **çok küçük listeler** için dizi oluşturmanın ek yükü NumPy'ı saf Python'dan yavaş yapabilir.

PYTHON

```python
import time
import numpy as np
liste = list(range(1_000_000))
dizi = np.arange(1_000_000)
t = time.perf_counter(); s1 = [x * 2 for x in liste]; liste_sure =
↪ time.perf_counter() - t
t = time.perf_counter(); s2 = dizi * 2; numpy_sure = time.perf_counter() - t
print('NumPy daha hızlı mı?', numpy_sure < liste_sure)
print('Sonuçlar aynı mı?', s1 == s2.tolist())
```

ÇIKTI

```
NumPy daha hızlı mı? True
Sonuçlar aynı mı? True
```

(Süreler bilgisayara göre değiştiği için burada yalnızca karşılaştırmanın sonucu yazdırıldı; terminalde süreleri kendin görebilirsin.)

N boyutlu dizi, eksenler ve kısıtlamalar (Slayt 9–10)

ndarray, aynı tipte elemanlardan oluşan ve negatif olmayan tam sayı tuple'larıyla indekslenen bir tablodur. Boyutlara **eksen** (axis) denir; 2 boyutlu dizide **axis 0 satırlar, axis 1 sütunlardır**.

ndarray'in üç "kısıtı" (**homojenlik, sabit boyut, dikdörtgen şekil**) aslında performans için bilinçli tercihlerdir: bitişik blok için her elemanın boyutu aynı olmalı ve toplam boyut oluşturulurken sabitlenmelidir. Karışık tipler verilirse NumPy hepsini ortak bir tipe çevirir.

PYTHON

```python
import numpy as np
m = np.array([[1, 2, 3], [4, 5, 6]])
print(m)
x = np.array([1, 2.5, 3])
print(x, x.dtype)
```

ÇIKTI

```
[[1 2 3]
 [4 5 6]]
[1.  2.5 3. ] float64
```

ndim, shape, size, dtype, itemsize (Slayt 11–13)

Öznitelik	Anlamı
`ndim`	eksen (boyut) sayısı
`shape`	her boyuttaki uzunluk (tuple); uzunluğu `ndim`'e eşittir
`size`	toplam eleman sayısı = shape elemanlarının çarpımı
`dtype`	eleman tipi (`int32`, `int64`, `float32`, `float64`...)
`itemsize`	eleman başına bayt (float64 → 64 bit = 8 bayt)

PYTHON

```python
import numpy as np
m = np.array([[1.5, 2.0, 3.0], [4.0, 5.0, 6.0]])
print('ndim:', m.ndim, '| shape:', m.shape, '| size:', m.size)
print('dtype:', m.dtype, '| itemsize:', m.itemsize)
print(np.array([1, 2], dtype=np.int32).itemsize)
```

ÇIKTI

```
ndim: 2 | shape: (2, 3) | size: 6
dtype: float64 | itemsize: 8
4
```

Dizi oluşturmak (Slayt 14–18)

Fonksiyon	Ne yapar?
`np.array(liste)`	mevcut veriden; tipi otomatik çıkarır, `dtype=` ile belirtilebilir
`np.zeros(shape)` / `np.ones(shape)`	sıfır / bir dolu (varsayılan **float64**)
`np.empty(shape)`	**başlatılmamış** (içerik rastgele; her elemana sonradan değer atanmalı)
`np.arange(start, stop, step)`	range gibi; **stop dahil değil**
`np.linspace(start, stop, num)`	**iki uç dahil** num eşit aralıklı değer
`np.eye(n)`	n×n birim matris
`np.random.rand(r, c)`	0–1 arası rastgele değerler

Boyut önceden biliniyorsa yer tutucu dizi oluşturmak, diziyi büyütmekten verimlidir. Ondalıklı adımlarda `linspace` tercih edilir: çıktı sayısını kesin kontrol eder ve kayan nokta sorunlarını önler.

PYTHON

```python
import numpy as np
print(np.array([1, 2, 3], dtype=np.float64))
print(np.zeros((2, 3)))
print(np.ones((2, 2)).dtype)
e = np.empty((2, 2))
print(e.shape, e.dtype)            # içerik rastgele olduğu için yazdırmıyoruz
print(np.arange(0, 10, 2), np.arange(5))
print(np.linspace(0, 1, 5))
print(np.eye(3))
np.random.seed(0)
print(np.random.rand(2, 2).round(3))
```

ÇIKTI

```
[1. 2. 3.]
[[0. 0. 0.]
 [0. 0. 0.]]
float64
(2, 2) float64
[0 2 4 6 8] [0 1 2 3 4]
[0.   0.25 0.5  0.75 1.  ]
[[1. 0. 0.]
 [0. 1. 0.]
```

ÇIKTI (DEVAM)

```
 [0. 0. 1.]]
[[0.549 0.715]
 [0.603 0.545]]
```

1 boyutlu indeksleme ve dilimleme (Slayt 19–20)

İndeksleme ve dilimleme veri filtrelemenin ve özellik mühendisliğinin temelidir. 1 boyutlu dizide sözdizimi listelerle aynıdır; indeks 0'dan başlar.

PYTHON

```python
import numpy as np
a = np.arange(10)
print(a[0], a[-1], a[2:5], a[::3])
```

ÇIKTI

```
0 9 [2 3 4] [0 3 6 9]
```

Çok boyutlu indeksleme (Slayt 21–22)

Tek köşeli parantez içinde virgülle: `dizi[satir, sutun]`. Tek başına `:` o eksenin **tamamı** demektir.

PYTHON

```python
import numpy as np
m = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(m[1, 2])
print(m[0, :], m[:, 1])
print(m[:2, 1:])
```

ÇIKTI

```
6
[1 2 3] [2 5 8]
[[2 3]
 [5 6]]
```

reshape ve 3 boyut (Slayt 23–24)

`reshape` elemanları değiştirmeden şekli değiştirir; yeni şeklin toplam eleman sayısı aynı olmalıdır. 3 boyutlu dizide indeks sırası `t[blok, satır, sütun]`'dur.

PYTHON

```python
import numpy as np
print(np.arange(12).reshape(3, 4))
t = np.arange(24).reshape(2, 3, 4)
print(t.shape)
print(t[0, :, :2])
```

PYTHON (DEVAM)

```
print(t[1, 2, 3])
```

ÇIKTI

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
(2, 3, 4)
[[0 1]
 [4 5]
 [8 9]]
23
```

View: listede kopya, NumPy'da pencere (Slayt 25–27)

Python listesini dilimlemek elemanların **kopyasını** üretir. NumPy'da basit dilimleme ise bir **view** döndürür: kendi verisi olmayan, orijinalle **aynı veri tamponuna** bakan bir nesne. View değişirse **orijinal de değişir**.

PYTHON

```
import numpy as np
liste = [0, 1, 2, 3, 4, 5]
parca = liste[1:4]
parca[0] = 9
print('liste:', liste)

a = np.arange(6)
v = a[1:4]
v[0] = 9
print('v:', v, '| a:', a)
print('aynı tamponu paylaşıyor mu?', np.shares_memory(a, v))
```

ÇIKTI

```
liste: [0, 1, 2, 3, 4, 5]
v: [9 2 3] | a: [0 9 2 3 4 5]
aynı tamponu paylaşıyor mu? True
```

Bu bir **performans tercihidir**: diziler gigabaytlarca olabilir; her dilim kopya olsaydı bellek kullanımı ve kopyalama yükü performansı çökertirdi. NumPy, listelerin "güvenli ama yavaş" kopyalama davranışı yerine hız ve bellek verimliliğini seçer.

Deep copy: .copy() (Slayt 28–29)

Orijinalden **tamamen bağımsız** bir dizi için `.copy()` açıkça çağrılmalıdır (**deep copy**).

PYTHON

```python
import numpy as np
a = np.arange(6)
c = a[1:4].copy()
c[0] = 9
print('c:', c, '| a:', a, '| paylaşım:', np.shares_memory(a, c))
```

ÇIKTI

```
c: [9 2 3] | a: [0 1 2 3 4 5] | paylaşım: False
```

Kaynaklar (Slayt 29): NumPy User Guide 2.2.0; "NumPy: the absolute basics for beginners" (numpy.org); H. Jani, "NumPy's SIMD-Friendly Design Boosts Performance Over Python Lists"; DATAQUEST, "Stop Using Python Lists and Start Using These Instead".

Sık yapılan hatalar

- `arange`**'in bitişi dahil sanmak:** `np.arange(10, 50)` 50'yi içermez; `np.arange(10, 51)` gerekir.
- **Uyumsuz** `reshape`**:** eleman sayısı tutmazsa `ValueError`.

PYTHON

```python
import numpy as np
np.arange(10).reshape(3, 4)
```

ÇIKTI

```
ValueError: cannot reshape array of size 10 into shape (3,4)
```

- **Var olmayan satıra erişmek** → `IndexError`.
- **Dilimin kopya olduğunu sanmak:** NumPy dilimini değiştirmek orijinali de değiştirir; bağımsız kopya için `.copy()`.
- `np.empty`**'nin sıfır döndürdüğünü sanmak:** içerik rastgeledir.
- `m(1, 2)` **yazmak:** indeksleme köşeli parantezle yapılır: `m[1, 2]`.

Kısa tekrar soruları

1 ndarray'i hızlı yapan iki bellek özelliği nedir?

2 `shape == (2, 3, 4)` olan dizinin `ndim` ve `size` değerleri nedir?

3 float64 dizide `itemsize` kaçtır?

4 `np.linspace(0, 1, 5)` ile `np.arange(0, 1, 0.25)` farkı nedir?

5 `m = np.arange(9).reshape(3, 3)` için `m[:, 0]` ve `m[1:, 1:]` nedir?

6 `a = np.arange(5); b = a[1:3]; b[:] = 0` sonrası `a` nedir? Bunu önlemek için ne yapılmalı?

Cevaplar

1 Homojen olması (her eleman aynı tip ve boyut) ve tek, bitişik bir bellek bloğunda saklanması.

2 `ndim = 3`, `size = 24`.

3 8 bayt.

4 `linspace` iki ucu da dahil eder: `[0. 0.25 0.5 0.75 1.]`; `arange` stop'u dahil etmez: `[0. 0.25 0.5 0.75]`.

5 `[0 3 6]` ve `[[4 5] [7 8]]`.

6 `[0 0 0 3 4]`; `b` bir view'dır. Önlemek için `b = a[1:3].copy()`.